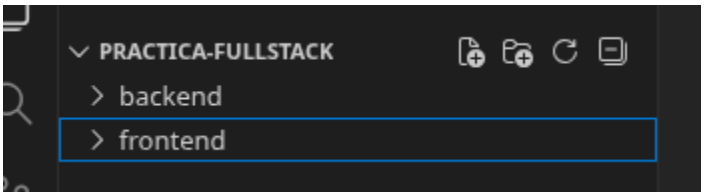
Documentación de la práctica FULLSTACK

Para el desarrollo de esta práctica se creó un proyecto fullstack dividido en dos partes principales: backend y frontend.

La estructura del proyecto quedó organizada de la siguiente manera:



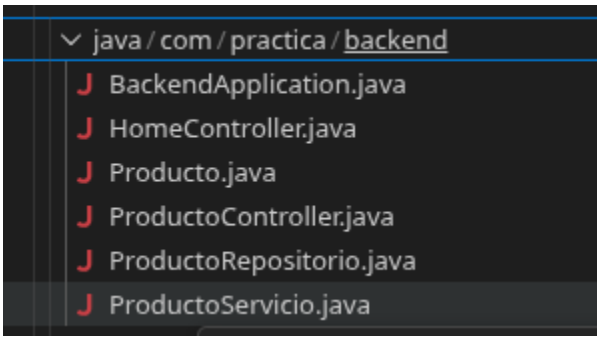
Cada carpeta contiene su propio entorno de desarrollo y dependencias necesarias para su ejecución.

El backend se desarrolló utilizando Spring Boot con Maven, mientras que el frontend se desarrolló con Angular.

2. Desarrollo del Backend (Spring Boot)

Se implementó lo siguiente:

- **Clase Producto,** representa la tabla en la base de datos.
- **Repositorio,** Interfaz que permite realizar operaciones CRUD con JPA.
- **Servicio,** Contiene la lógica de la gestión de productos.
- **Controller,** Expone los endpoints de la API.



Campos de la entidad Producto:

- id (clave primaria)
- nombre
- precio
- categoría

- existencias

Dependencias utilizadas:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Validation
- Lombok (opcional)

Ejecución del proyecto

Backend:

1. Entrar a la carpeta backend
2. Ejecutar: mvn spring-boot:run

Frontend:

1. Entrar a la carpeta frontend
2. Ejecutar: ng serve

El sistema estará disponible en: http://localhost:4200

Base de datos:

Se utilizó MariaDB como sistema gestor de base de datos. Para la conexión se empleó el driver MySQL Connector debido a su compatibilidad con MariaDB.

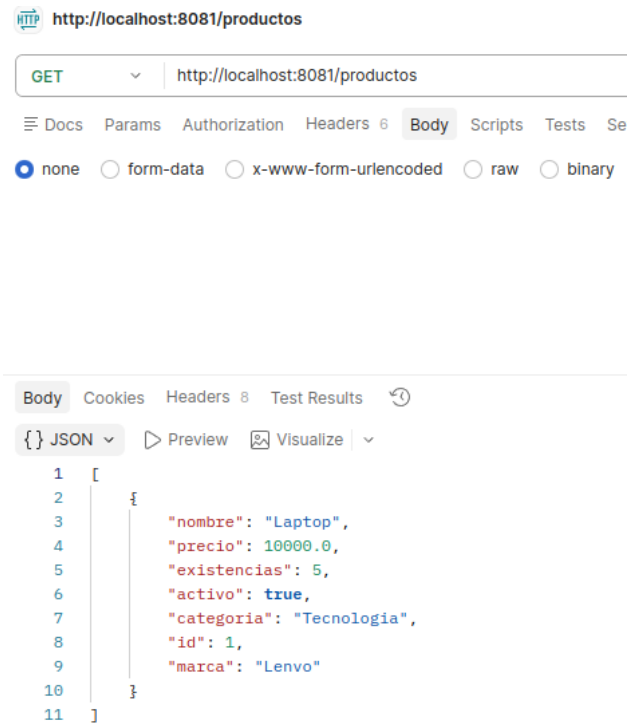
La configuración se realizó en el archivo application.properties del backend.

3. Endpoints de la API

Se implementaron los siguientes endpoints para la gestión de productos, las pruebas se realizaron en Postman:

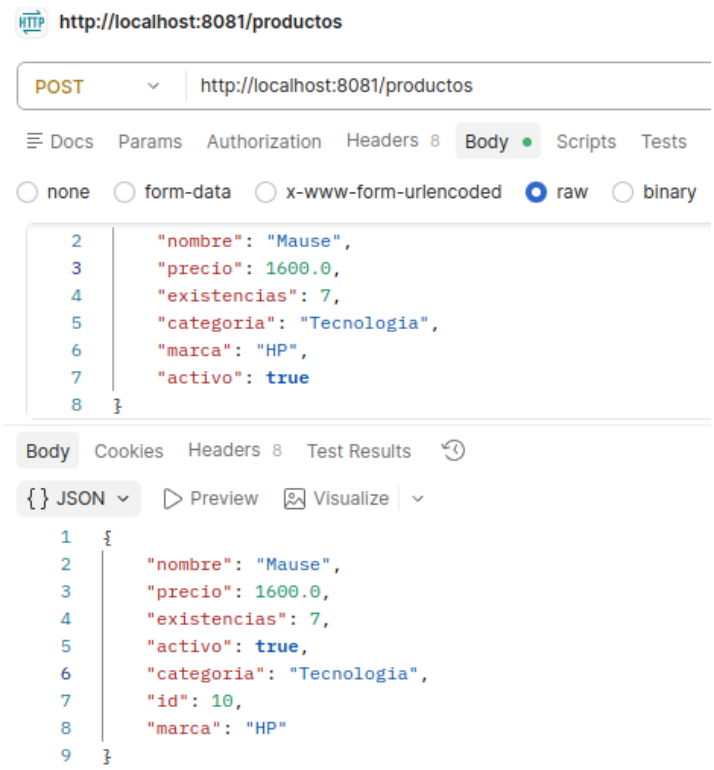
Obtener todos los productos guardados en la base de datos

- GET /productos



Crear un producto

- POST /productos



Actualizar un producto

- PUT /productos/{id}

http://localhost:8081/productos/10

PUT http://localhost:8081/productos/10

Docs Params Authorization Headers 8 Body Scripts Tests

none form-data x-www-form-urlencoded raw binary

```
2    "nombre": "Mause",
3    "precio": 1600.0,
4    "existencias": 7,
5    "categoria": "Tecnologia",
6    "marca": "HP",
7    "activo": true
8  }
```

Body Cookies Headers 8 Test Results

{ } JSON Preview Visualize

```
1  {
2    "nombre": "Mause",
3    "precio": 1600.0,
4    "existencias": 7,
5    "activo": true,
6    "categoria": "Tecnologia",
7    "id": 10,
8    "marca": "HP"
9  }
```

Eliminar un producto

- DELETE /productos/{id}

4. Desarrollo del Frontend (Angular)

El frontend se desarrolló con Angular para consumir la API del backend.

Se implementaron:

- ✓ Servicios para consumir la API
- ✓ Componentes para mostrar los productos
- ✓ Formularios para agregar y editar productos
- ✓ El frontend se conecta al backend mediante peticiones HTTP.

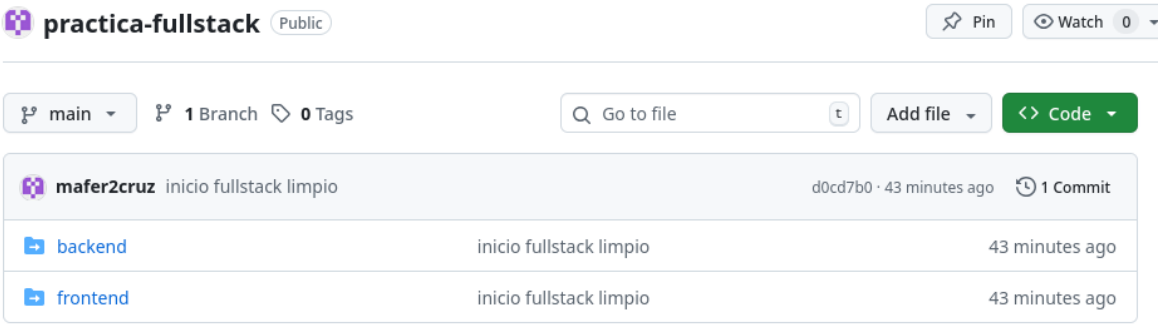
5. Flujo de trabajo con Git

Se realizaron pruebas utilizando Postman para verificar el correcto funcionamiento de la API.

Se validaron las siguientes operaciones:

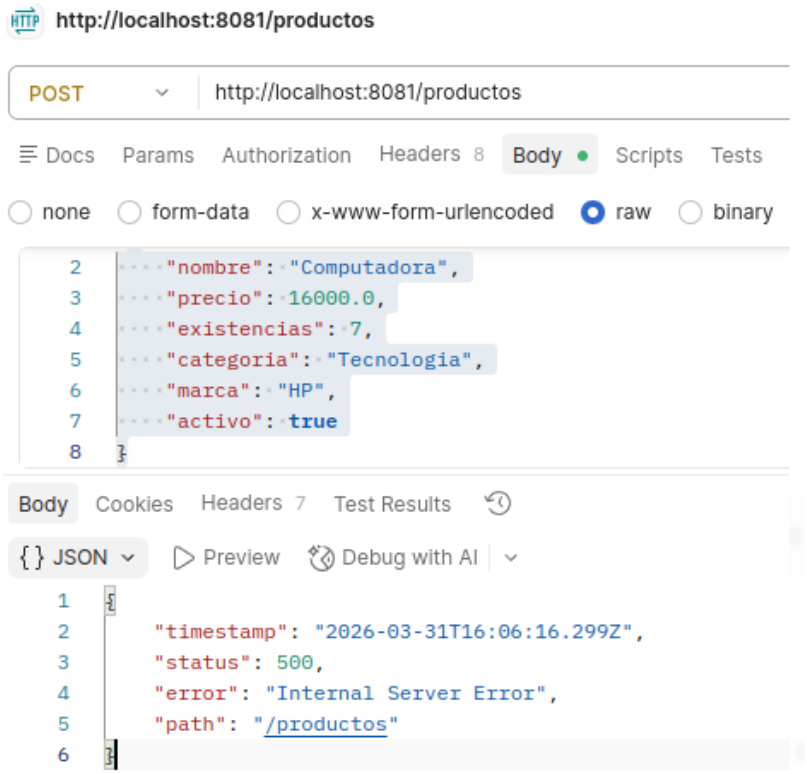
- Creación de productos
- Consulta de productos
- Actualización de datos
- Eliminación de registros

También se verificó la integración entre frontend y backend.



7. Debug y resolución de errores

Error 1. Error al guardar un producto con el mismo nombre



Error 2. Fusión en archivos de configuración

Durante la integración de los módulos, Git lanzó un error de CONFLICTO (agregar/agregar) en el archivo .gitignore.Tanto el proyecto de Spring Boot como el de Angular generaron automáticamente sus propios archivos .gitignore, y Git no podía decidir automáticamente cuál conservar al intentar fusionar las ramas.

Error 3. Error de validación de campos vacíos en el Frontend (Angular)

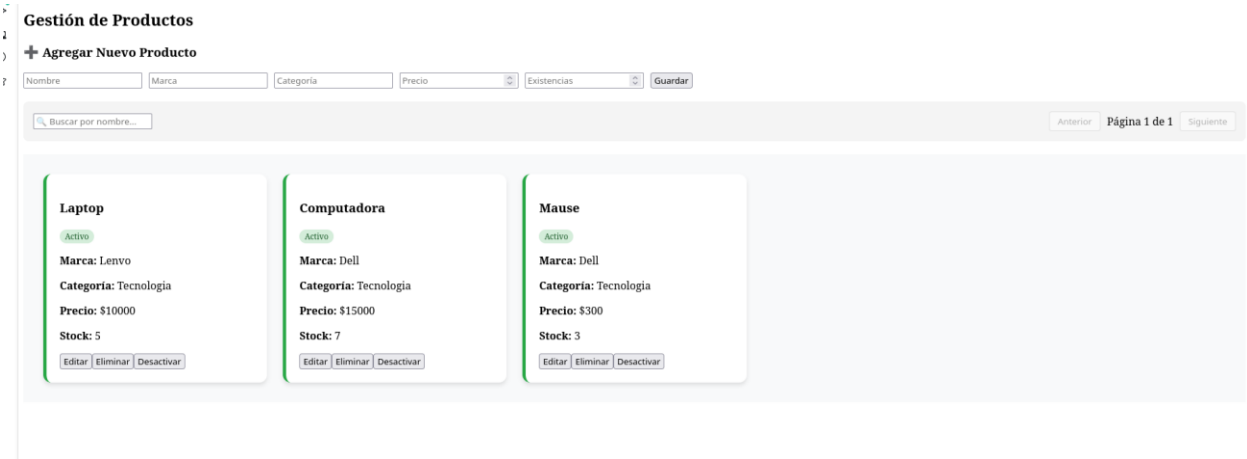
El formulario de login permitía enviar peticiones nulas al servidor, causando errores innecesarios en el backend. Falta de lógica de control en el componente de TypeScript para validar los estados de los campos de usuario y contraseña antes de ejecutar la función login().

Resultados:

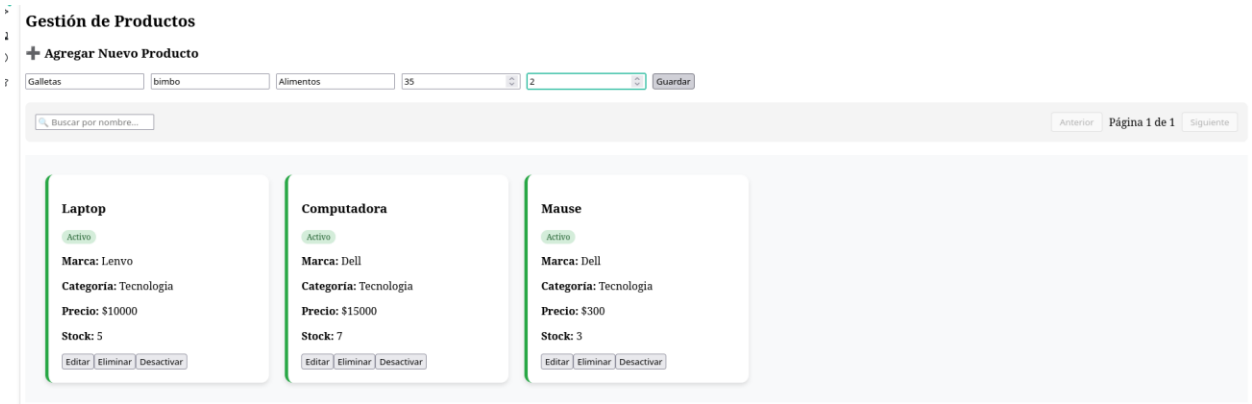
Inicio de sesión

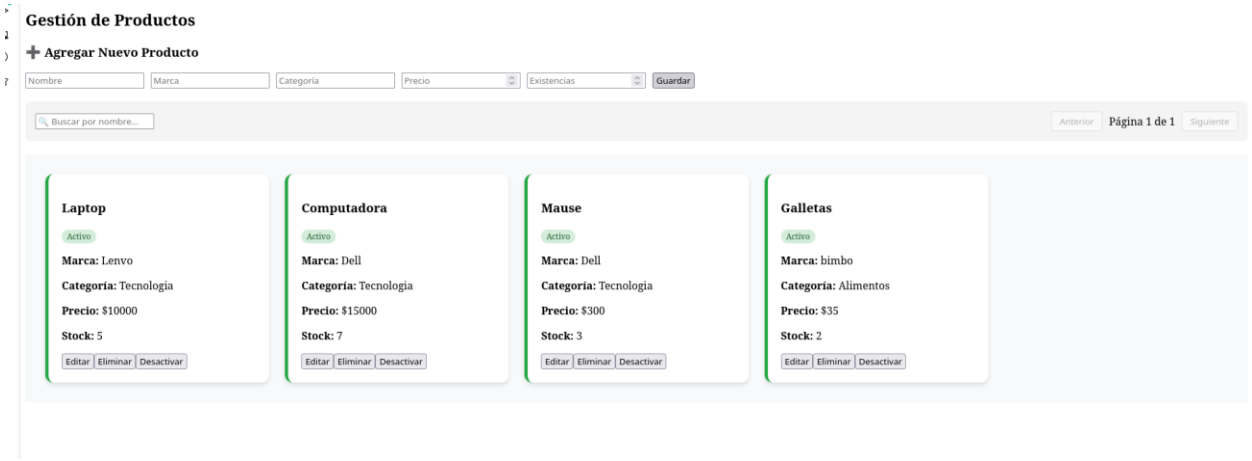
Usuario: admin
pasword: 1234

Vista principal

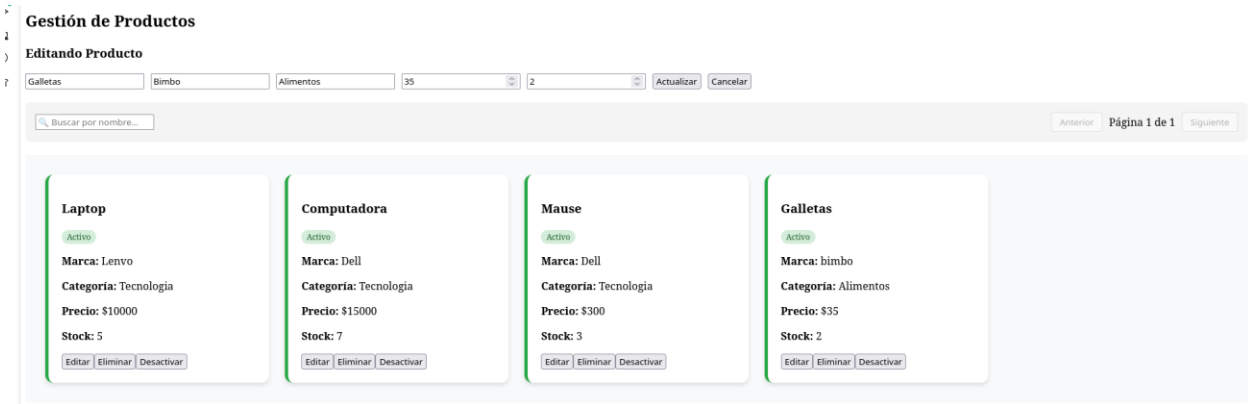


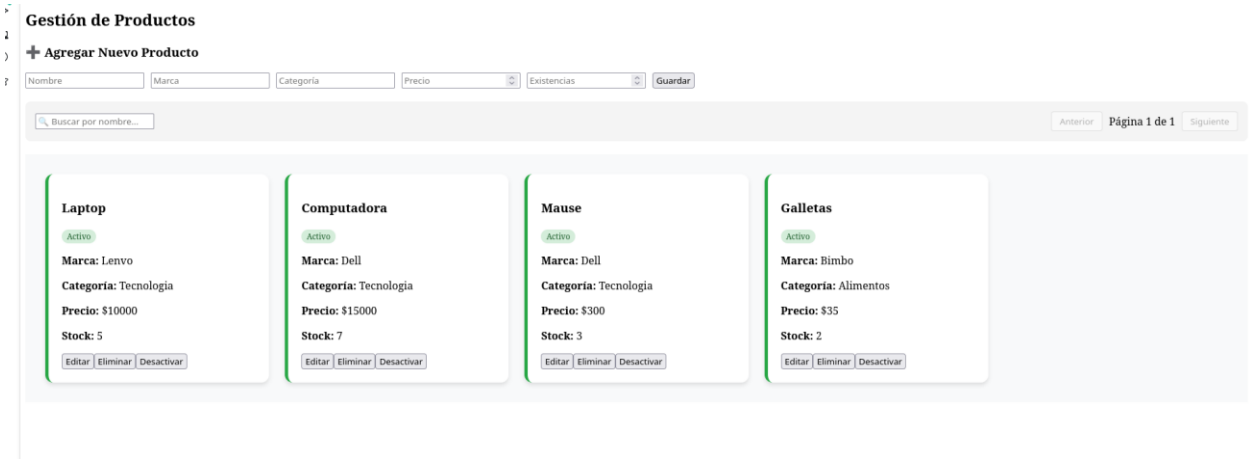
Agregar productos:



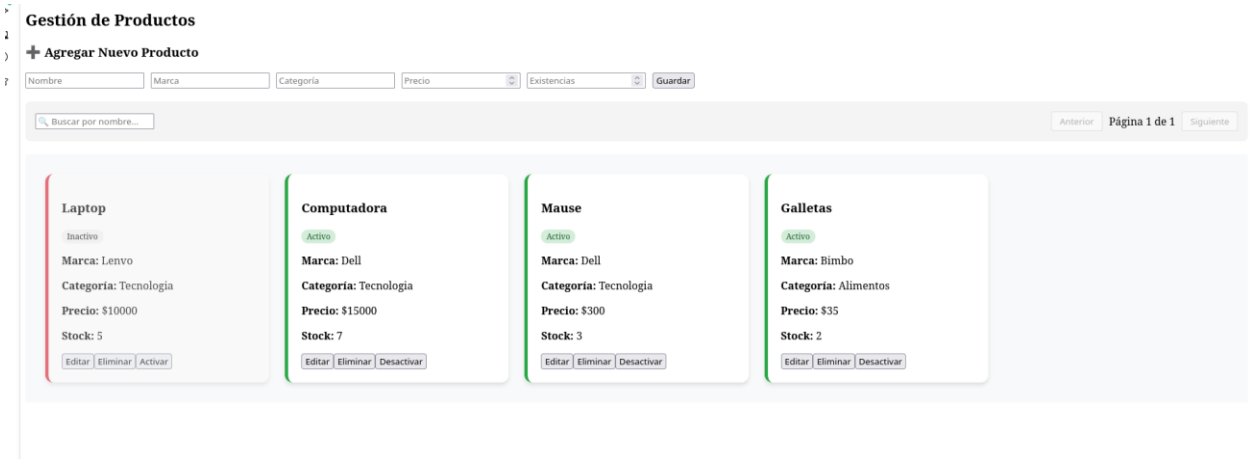


Editar producto:

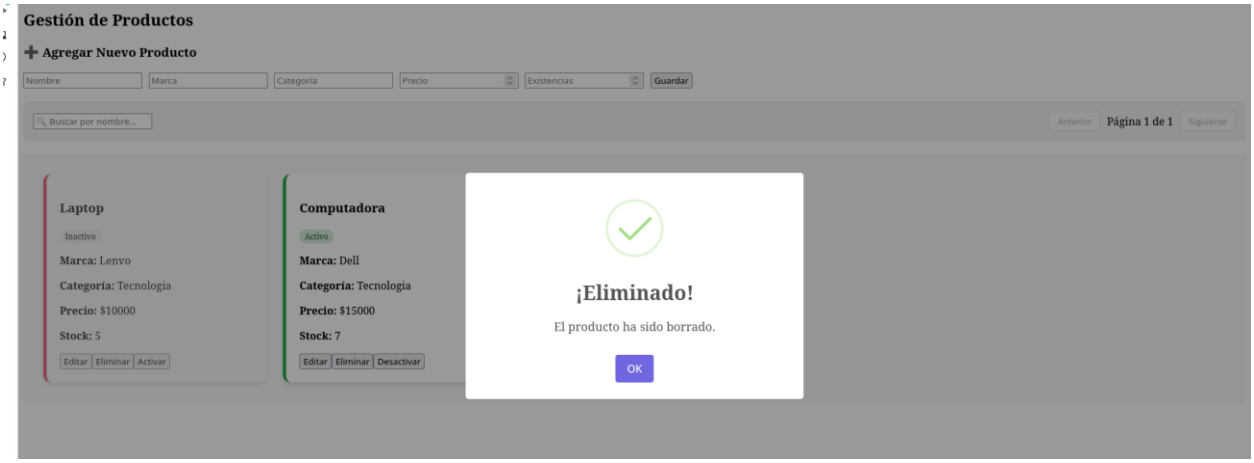




Activar o desactivar producto



Eliminar producto:



Conclusión

Se logró desarrollar un sistema fullstack funcional utilizando Spring Boot y Angular, implementando operaciones CRUD completas y validaciones tanto en backend como en frontend. Se resolvieron distintos errores durante el desarrollo, lo que permitió comprender mejor el flujo de integración entre tecnologías.